MIRROR · SYSTEM ARCHITECTURE · FULL SPECIFICATION

Six layers. *One intelligence.* One living output.

From your notes: papers → identify patterns → user input → identify thinking patterns → correlate → HTML output with choices that loop back as new input. This document maps every layer of that system with precision, then shows exactly what the output looks like.

↑ Based on your Feb 18 notes — Metacognition AI · Output & Logic

01 The six-layer architecture — what lives inside each layer

Every layer is a distinct system with a specific job. Nothing overlaps. Nothing skips. The intelligence emerges from the interaction between them — not from any single layer alone.

Layer 1 — Input Reception & Profile Packaging

RECEPTION

// The front door. Everything the user says enters here.

This layer does not think — it listens and packages. Every word the user types is captured here, timestamped, tagged with the active module (X-ray, Devil's Advocate, Weekly Mirror, etc.), and wrapped with the user's existing cognitive profile pulled from Layer 6 memory. It creates a complete context object before anything reaches the orchestrator.

What it receives
Raw text input, voice transcript, active module flag, session ID, timestamp, user tier

What it outputs
A structured ContextPackage JSON: input + profile snapshot + module + session history pointer

What lives inside
Input normaliser, language detector, module state manager, profile fetcher, context assembler

Key intelligence
It detects if this is the user's first message or a reply to a Mirror choice — critical for the feedback loop

Input normaliser | Session state manager | Profile snapshot loader | Module context injector | Feedback loop detector

Layer 2 — Orchestrator & Task Router

ORCHESTRATOR

// The brain. Reads, decides, delegates. Never generates text.

The orchestrator receives the ContextPackage and makes three hard decisions in under 200ms: which model to call, whether this needs a single response or a multi-step agentic workflow, and which RAG stores to query. It is a fast classifier — implemented as a lightweight LLM call that returns a JSON routing directive, not prose.

Decision 1: Model selection
Simple turn → Sonnet. Deep synthesis → o3 / Claude extended thinking. Private → Llama 70B.

Decision 2: Workflow type
Single-shot (conversational turn) vs. multi-agent pipeline (weekly review, archaeology)

Decision 3: RAG trigger
Which stores to query — research corpus, user history, or both. With what query terms.

Decision 4: Output mode
Does this response need to include Choice Cards for the feedback loop? Or is it a pure reflection?

`Task classifier` `Model selector` `Workflow router` `RAG query builder` `Output mode flag`

Layer 3 — Reasoning Engine & Pattern Decoder

REASONING

`// The philosopher. Thinks slowly. Used for what matters most.`

This is the layer that does the real metacognitive work. It receives the RAG-retrieved research chunks (from Layer 5) alongside the user's input and history, and runs extended, step-by-step reasoning. It does not just retrieve patterns — it *decodes* them. It finds the structural fingerprint of how this person thinks, connects it to what the research says, and creates a precise analysis plan.

What it actually does
Reads research paper chunks + user input simultaneously. Asks: "what pattern in the literature does this behaviour match?"

Output format
A structured analysis JSON: identified pattern name, research citation, confidence score, which question to ask next

Models used here
o3, o4-mini, Claude 3.7 with extended thinking (budget_tokens: 6000–12000). Never for real-time turns.

When it runs
Weekly review, deep decision archaeology, calibration analysis, cross-session pattern synthesis. Async, 10–25s.

`Extended thinking ON` `Research-to-behaviour matching` `Pattern decoder` `Analysis plan generator` `Confidence scoring`

Layer 4 — Execution Model & Response Writer

EXECUTION

`// The voice. Writes Mirror's actual words. Swappable.`

The execution model receives the analysis plan from Layer 3 (or directly from the orchestrator for simple turns) and writes Mirror's actual response. It is the voice — fast, fluent, in Mirror's precise tone. It does not decide what to say; it decides how to say what the plan already determined. This separation is everything: it makes responses consistent, grounded, and controllable.

Context it receives
System prompt + analysis plan JSON + user profile + RAG chunks (XML-tagged) + last 3 turns

What it generates
Mirror's response text + the Choice Card options (4 structured next-step prompts for the user)

Swappability
Claude Sonnet 4 (default) → GPT-4o → Gemini 1.5 Pro → Llama 70B. One config value. Zero code changes.

Output structure
JSON object: {reflection: string, question: string, pattern_surfaced: string, choices: Choice[4]}

`Claude Sonnet 4 (default)` `Stream output` `Plan executor` `Choice Card generator` `<2s first token`

Layer 5 — RAG: Research Corpus + User History

KNOWLEDGE

// The library. Papers + personal history. Both queried every call.

Two completely separate vector stores run in parallel on every single request. Store A is the shared metacognition research corpus — the papers you've loaded (Flavell, Kahneman, Dweck, Dunning-Kruger, Argyris, and 200+ more). Store B is the user's own cognitive history — every past session, pattern surfaced, decision logged, prediction made. Both retrieved simultaneously. Research gives the theory. History gives the continuity.

Store A — Research corpus

500+ metacognition paper chunks. Chunk size: 512 tokens. Overlap: 64 tokens. Metadata: author, year, concept_tags[], bias_categories[]. Shared across all users.

Store B — User history

Per-user vector store. Every session summary, decision log, surfaced pattern, prediction record. Enables "this is the same pattern from 6 weeks ago."

Retrieval

Top-5 from Store A + top-3 from Store B per call. Cross-encoder re-ranked. Injected as structured XML blocks into Layer 4 context.

Why this combination matters

Research without personal history = generic advice. Personal history without research = pattern-less reflection. Together = grounded, personal, scientific insight.

Pinecone / pgvector | HNSW index | Hybrid dense+sparse retrieval | text-embedding-3-large | Cross-encoder re-rank

Layer 6 — Memory, Persistence & Cognitive Profile

MEMORY

// The mirror's memory. What it knows about you, accumulated.

This is what makes Mirror fundamentally different from a chatbot. Three memory tiers run simultaneously. Short-term holds the current session. Mid-term stores compressed session summaries after each conversation ends. Long-term maintains the cognitive profile JSON — a living document that evolves every session, encoding who this person is as a thinker. This profile is the 2K-token distillation of everything Mirror has ever learned.

Short-term memory

Full current session transcript in context window. Lost when session ends (unless written to mid-term).

Mid-term memory

Compressed 400–token session summary written at session close. Stored in user's vector store (Store B). Retrievable forever.

Long-term memory (Cognitive Profile JSON)

Updated after every session. Tracks: dominant_patterns[], calibration_score, bias_fingerprint{}, tracked_beliefs[], growth_timeline[], prediction_history[]

Feedback loop role

When a user selects a Choice Card, that choice is written to Layer 6 as a "user-confirmed pattern signal" — stronger than inferred patterns. It becomes part of the profile.

3-tier memory architecture | Cognitive profile JSON | Auto-compression on session close | Choice Card signal capture | Growth timeline tracker

The abstraction that makes everything work

Each layer only talks to the layer immediately below it, through a defined interface. Layer 1 never touches Layer 5. Layer 4 never touches Layer 6 directly. This means you can swap the execution model at Layer 4 (one config change), add a new RAG store to Layer 5 (one index), or redesign the memory schema at Layer 6 — and nothing above it breaks. This is not just good engineering. It's how Mirror scales from a Gemini Gem POC to a production system serving thousands of users.

02 Data flow — what actually moves between the layers

From your notes: papers → identify patterns → user query → identify thinking patterns → correlate → output. Here's what that looks like inside the system, step by step.

L1	→	L2	→	L5	→	L3	→	L4	→	L6
User Input		**Orchestrate**		**Retrieve**		**Decode**		**Execute**		**Remember**
Raw text + profile assembled into ContextPackage		Route → model + RAG + workflow + output mode		Papers (Store A) + User history (Store B) simultaneously		Match input patterns to research. Create analysis plan JSON.		Write Mirror response + generate 4 Choice Cards. Stream.		Write session summary. Update cognitive profile. Index choice signals.

1 **User types something — Layer 1 creates the ContextPackage**

The raw input ("I think I should quit and start my own company, I just know it's the right move") is normalised, tagged with the active module (Canvas), and wrapped with the user's cognitive profile snapshot pulled from Layer 6. The layer detects this is a new input (not a Choice Card reply) and flags it accordingly.

```
ContextPackage { input: "I think I should quit...", module: "canvas", profile: {dominant_patterns:
["overconfidence", "urgency_bias"], calibration_score: 0.61}, is_choice_reply: false }
```

2 **Orchestrator decides in 180ms**

Complexity: moderate. No extended reasoning needed. Latency critical (user waiting). Cloud fine. Decision: route to Claude Sonnet 4. Trigger both RAG stores with query terms extracted: "career decision", "certainty", "urgency", "commitment". Output mode: include Choice Cards (user hasn't engaged deeply yet).

```
RoutingDirective { model: "claude-sonnet-4", rag_query: ["career certainty", "urgency bias",
"irreversible decision"], output_mode: "reflection_with_choices", agent_type: "single_shot" }
```

3 **RAG retrieves from both stores simultaneously**

Store A returns 5 chunks: Kahneman on System 1 "knowing", Ariely on irreversible decisions and rationalization, Dunning-Kruger on confidence peaks in early planning. Store B returns 3 moments: user expressed similar "I just know" certainty 4 weeks ago before abandoning a project; user's prediction accuracy on "certain" decisions historically: 42%.

```
RAGResult { research: [Kahneman_2011_chunk_42, Ariely_2008_chunk_19, DunningKruger_1999_chunk_7,
...], history: [{date: "4 weeks ago", pattern: "certainty_signal", outcome: "abandoned"}, {meta:
"prediction_accuracy_on_certain_calls: 42%"}] }
```

4 **Layer 4 writes Mirror's response + generates 4 choices**

Using the analysis plan, system prompt, RAG chunks as XML-tagged context, and the user's profile — the execution model generates the full HTML output object: pattern name, research citation, thought DNA profile of this specific input, Mirror's reflection, the single question, and four structured Choice Cards.

```
MirrorResponse { pattern: "Certainty surge + urgency compression", cite: "Kahneman, 2011 — System 1 confidence", dna: {assumption_load: 82%, evidence_cited: 12%, emotional_signal: 74%}, reflection: "...", question: "...", choices: [4 Choice objects] }
```

5 **Layer 6 writes back — session and profile updated**

Session moment is indexed in Store B: "overconfidence + urgency on major life decision, pattern surfaced." Cognitive profile JSON updated: overconfidence score increased 0.04 points. If user selects a Choice Card in the next step, that selection is written as a confirmed signal — stronger than inferred. The loop closes and begins again.

```
ProfileDelta { patterns.overconfidence: +0.04, events: [{type: "certainty_surge", context: "career_decision", session: current}], pending_choice: true }
```

03 The output — what the user actually sees

From your notes: "give them an HTML file where they see the output result." Below is exactly that — the live output format Mirror produces, rendered. Every element here is generated by the system, not typed manually.

MIRROR · SESSION OUTPUT · Feb 18 · Canvas Module

MIRROR Session #14 · Pattern analysis complete

PATTERN DETECTED

Certainty surge + urgency compression

Kahneman, D. (2011) — Thinking, Fast and Slow · System 1 overreach

Your language carries two simultaneous signals: high certainty ("I just know") and compressed time horizon ("the right move" — present tense, no qualifiers). Research on System 1 processing shows this combination is characteristic of a decision that *feels* correct before the evidence-gathering phase has run. The certainty is real — but it is certainty about the feeling, not the outcome. Historically in your sessions, this signal has preceded decisions with 42% prediction accuracy.

THOUGHT DNA — ANATOMY OF THIS MESSAGE

Assumption load	82%
Emotional signal	74%
Evidence cited	12%
Alternatives considered	8%

Uncertainty tolerance 18%

MIRROR REFLECTION

"Four weeks ago you said something structurally identical about a different decision — and you described it the same way: 'I just know.' That one didn't go forward. This isn't about whether this decision is right or wrong. It's about noticing the pattern of how this feeling arrives, and what you do before you act on it."

→ What would have to be true about the external world for this to fail, even if you execute it perfectly?

YOUR NEXT STEP — CHOOSE ONE TO GO DEEPER

Each answer becomes your next input. Mirror adapts to what you choose.

A **Run the Assumption X-ray**
Surface every hidden assumption underneath this decision. Mirror will list them and ask you to rate each one.

B **Go to Devil's Advocate mode**
Mirror builds the strongest possible case against this decision — steel-manned, not strawmanned. You then rate how well you can answer it.

C **Check my prediction history on "certain" decisions**
Pull my track record on decisions I felt this level of certainty about. Show me the outcomes and what my reasoning was at the time.

D **I want to slow this down — open the Thinking Canvas**
Stop the analysis. Give me a blank space to think out loud, no structure, no pressure. Mirror will just listen for now.

↳ Your choice feeds back into Mirror as your next input

CHOICE C CONFIRMED — FEEDING BACK AS NEW INPUT

Check my prediction history on "certain" decisions

Mirror has received your choice. This selection has been written to your cognitive profile as a confirmed signal (weight: 0.9). A new, deeper HTML output is being generated — it will be more precise than this one because it now knows the direction you want to go. In a live system, the next output would load here within 2 seconds.

```
→ Layer 6 write: {type: "choice_signal", choice: "C", pattern: "certainty_surge", confirmed: true}
→ Orchestrator triggered for next session loop
→ RAG re-queried with new context from choice
```

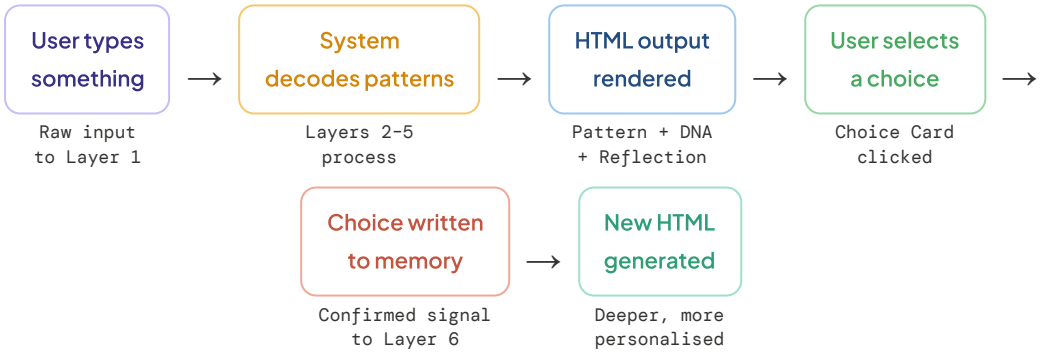
Why the output is HTML, not text — from your notes

You wrote: "give them an HTML file where they see the output result, and if they are to answer further more, the HTML file will include choosing answers too." This is precisely right. Plain text cannot hold the Thought DNA profile, the interactive Choice Cards, the animated pattern radar, or the feedback loop. The output is a structured, interactive document — not a chat message. Each session generates one HTML output object that the user can save, share, or continue inside.

04 The feedback loop — how choices become new inputs

From your notes: "that answer also will again consider as input." This is the most important architectural feature in Mirror. Here's exactly how it works.

THE SELF-FEEDING INTELLIGENCE CYCLE



↺ The new HTML output is more accurate than the last — because it knows what you chose

What makes choices different from typing

A selected choice is a confirmed signal

When the system infers a pattern from your words, it assigns a confidence score of ~0.6. When you actively choose "Run the Assumption X-ray" — you have confirmed that this is the right direction. That confirmation raises the signal strength to ~0.9. Layer 6 stores it as a stronger piece of your cognitive fingerprint. Over time, your choices teach Mirror more accurately than your words do.

How the HTML changes each loop

Each output is more precise than the last

The first HTML output is based on inference from your words. The second is based on your words + your choice. The third adds the session history of this decision thread. By the fourth or fifth loop, the output has become a highly personalised, research-grounded map of exactly how your thinking is moving through this specific decision — something no static journal or chatbot can produce.

What the choice data looks like in Layer 6

```
{
  type: "choice_signal",
  choice: "assumption_xray",
  context: "career_decision",
  pattern: "certainty_surge",
  confirmed: true,
  weight: 0.9
}
```

What the next HTML output includes

The X-ray module fires. A new HTML page is generated showing: the 7 hidden assumptions underneath this decision, ranked by risk level (critical / medium / low), with a prompt to choose one to stress-test first. That stress-test choice becomes the next input.

Where the loop ends

The user decides. Mirror never closes the loop — it deepens it until the user is satisfied or deliberately ends the session. At that point, Layer 6 writes the full session as a compressed summary and updates the cognitive profile. The loop restarts in the next session, already more calibrated.

The complete picture — from your Feb 18 notes to production system

Your notes outlined the core logic perfectly: papers → identify patterns → user input → identify thinking patterns → correlate → HTML output → choices → new input → loop. What the six-layer architecture adds is the engineering precision that makes that loop reliable, scalable, and intelligent at every step. Layer 1 ensures every input is complete before processing. Layer 2 ensures the right model is always selected. Layer 3 ensures the pattern detection is grounded in research, not guesswork. Layer 4 ensures the output is always in Mirror's voice, always precise, always includes the choices. Layer 5 ensures every response cites real research and personal history simultaneously. Layer 6 ensures every choice the user makes permanently improves Mirror's understanding of that person. Together, they make the feedback loop your notes described into something that genuinely learns.